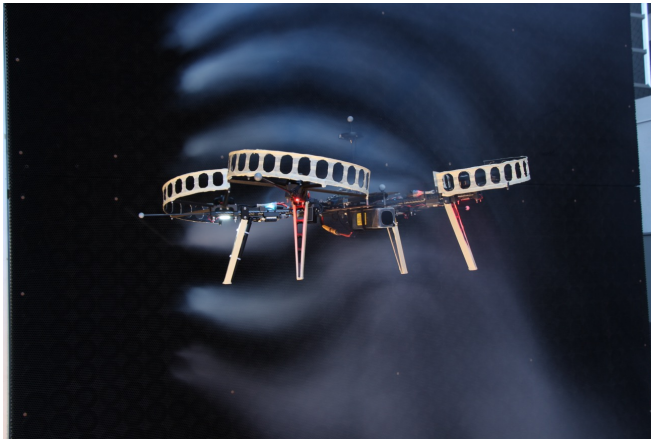
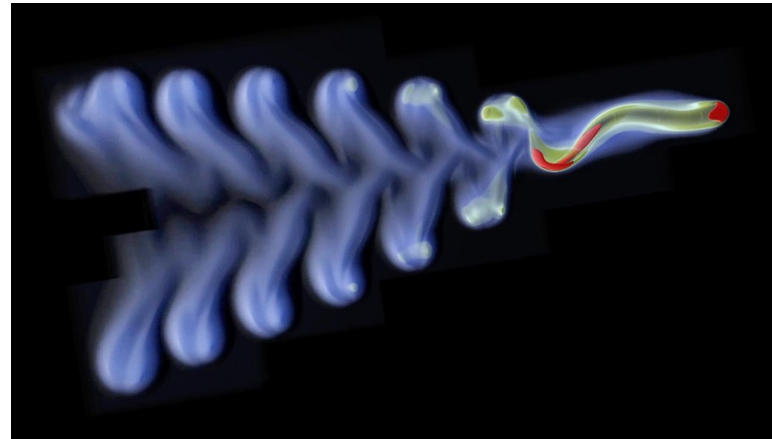


Introduction to Model-based RL and Control

$$\dot{x} = \frac{dx}{dt} = f(x, u) \quad x_{t+1} = f(x_t, u_t)$$



Aerodynamics in wind, *Neural-Fly*



Fluid dynamics, MIT van Rees Lab



Offroad vehicle dynamics, UW Racer team

Instructor: Guanya Shi

Recording Reminder

- ☐ This session will be audio recorded for educational use by other students in this course.

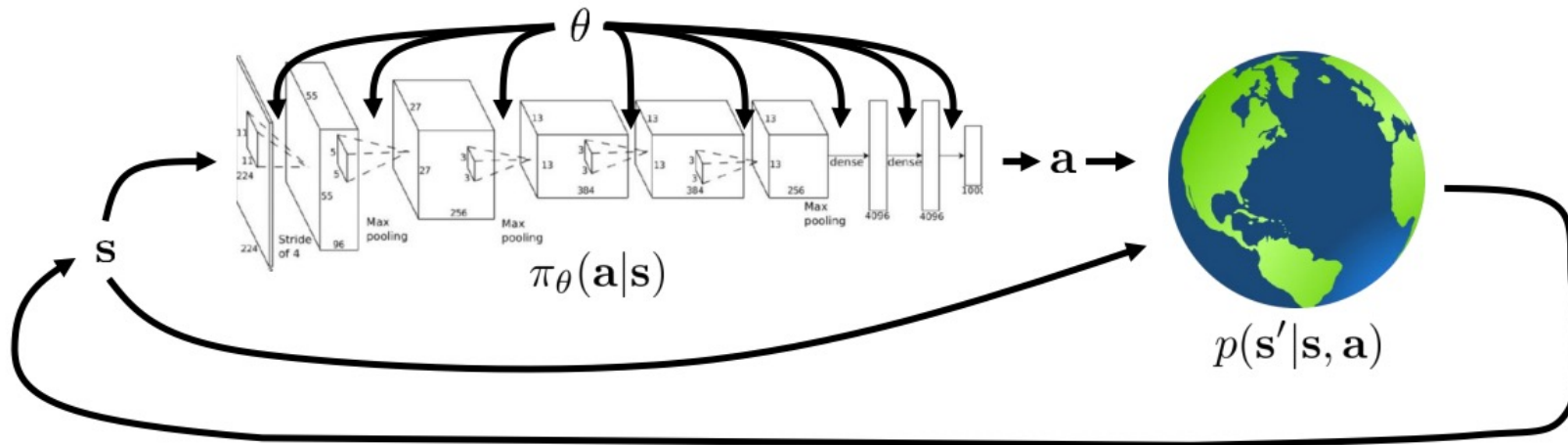
Course Structure

Latest schedule: <https://16-831-s24.github.io/lectures>

- ☐ Topic group 1: Course introduction: What is robot learning and why?
- ☐ Topic group 2: Machine/Deep learning refresher
- ☐ Topic group 3: Imitation learning
- ☐ Topic group 4: Model-free reinforcement learning
- ☒ Topic group 5: Model-based reinforcement learning (one guest lecture)
- ☐ Topic group 6: Reinforcement learning from offline data (one guest lecture)
- ☐ Topic group 7: Bandit, preference-based learning, exploration
- ☐ Topic group 8: Specialized topics (one guest lecture)
- ☐ Project presentation & poster session

Recap: Model-free RL

- ❑ Objective of RL (for any RL algorithm, either model-based or model-free):



$$\underbrace{p_{\theta}(s_1, a_1, \dots, s_T, a_T)}_{\pi_{\theta}(\tau)} = p(s_1) \prod_{t=1}^T \pi_{\theta}(a_t | s_t) p(s_{t+1} | s_t, a_t)$$

model-free RL assumes it is unknown
(don't even attempt to learn it)

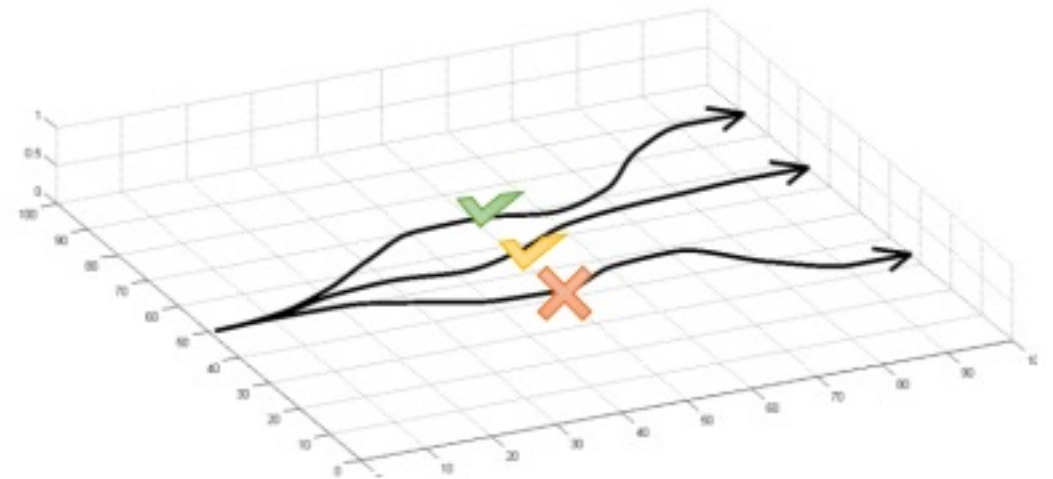
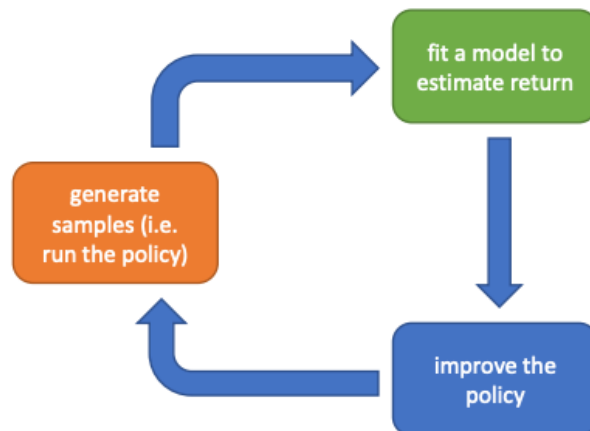
$$\theta^* = \arg \max_{\theta} E_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(s_t, a_t) \right]$$

Recap: Model-free RL

- ❑ Example: REINFORCE (the basic policy gradient algorithm)
 - Directly interact with the environment to improve the policy
 - Trail and error: make “good stuff” more likely and “bad stuff” less likely
 - No model in the algorithm

REINFORCE algorithm:

1. sample $\{\tau^i\}$ from $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$ (run the policy)
2. $\nabla_\theta J(\theta) \approx \sum_i \left(\sum_t \nabla_\theta \log \pi_\theta(\mathbf{a}_t^i|\mathbf{s}_t^i) \right) \left(\sum_t r(\mathbf{s}_t^i, \mathbf{a}_t^i) \right)$
3. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$



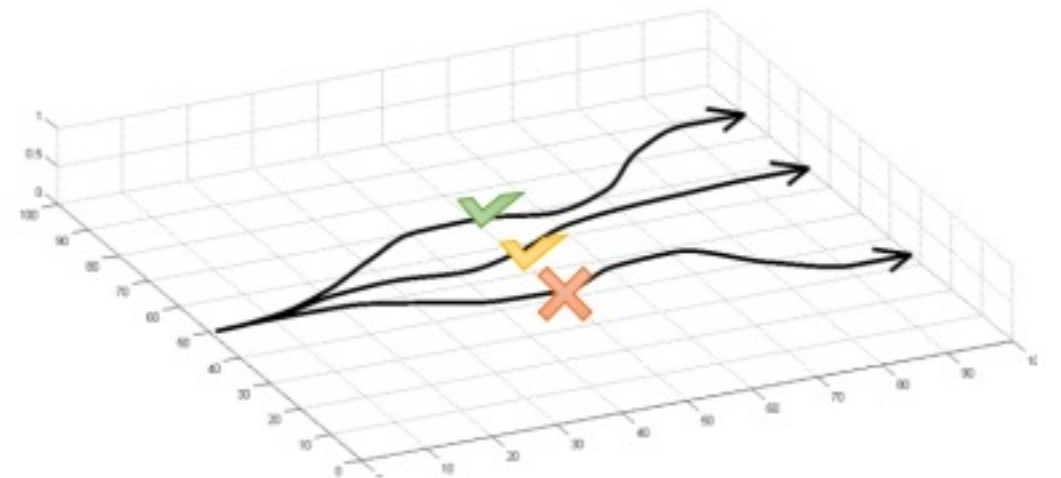
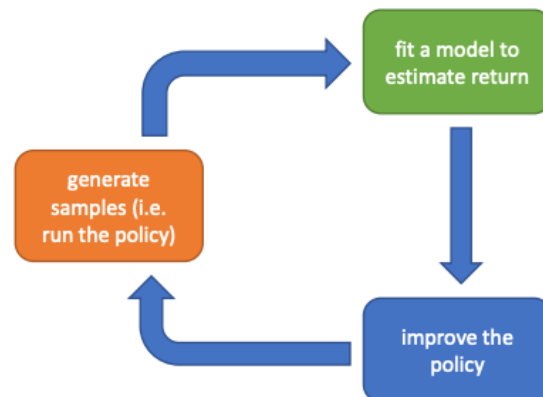
Recap: Model-free RL

❑ Pros and cons of “trail and error”

- High variance
- Poor sample complexity
- Hard to analyze
- Safety dilemma: you have to try before knowing its wrong/fatal
- Very general and no structured assumption - given enough time, good performance!
- Too general so may not optimal for some specific problems

REINFORCE algorithm:

-
1. sample $\{\tau^i\}$ from $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$ (run the policy)
 2. $\nabla_\theta J(\theta) \approx \sum_i (\sum_t \nabla_\theta \log \pi_\theta(\mathbf{a}_t^i|\mathbf{s}_t^i)) (\sum_t r(\mathbf{s}_t^i, \mathbf{a}_t^i))$
 3. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$



What if We Know Transition Dynamics?

- ❑ Often we do know the dynamics
 - Games (e.g., Atari, Go, chess, DOTA2)
 - Derived from the first principle (e.g., rigid-body dynamics)
 - Simulated environments (robot simulators)
- ❑ Often we can learn the dynamics
 - Offline: system identification (fit unknown parameters of a known model)
 - Offline: model learning (fit a general model to observed data)
 - Online: online learning (online fit unknown parameters)
- ❑ Knowing the model often makes things better/easier!



Model-based RL and Control

- ❑ A high-level recipe: Get the transition dynamics, then figure out how to choose actions!
- ❑ Dynamics model and notations:
 1. Deterministic continuous-time dynamics model
 2. Deterministic discrete-time dynamics model
- ❑ 1. and 2. both have stochastic versions
- ❑ $x \in \mathbb{R}^n$: state of the robot; $u \in \mathbb{R}^m$: control input

$$\begin{aligned} 1. \dot{x} &= \frac{dx}{dt} = f(x, u) \\ 2. x_{t+1} &= f(x_t, u_t) \end{aligned}$$

s : state
 a : action
 r : reward



Richard E. Bellman

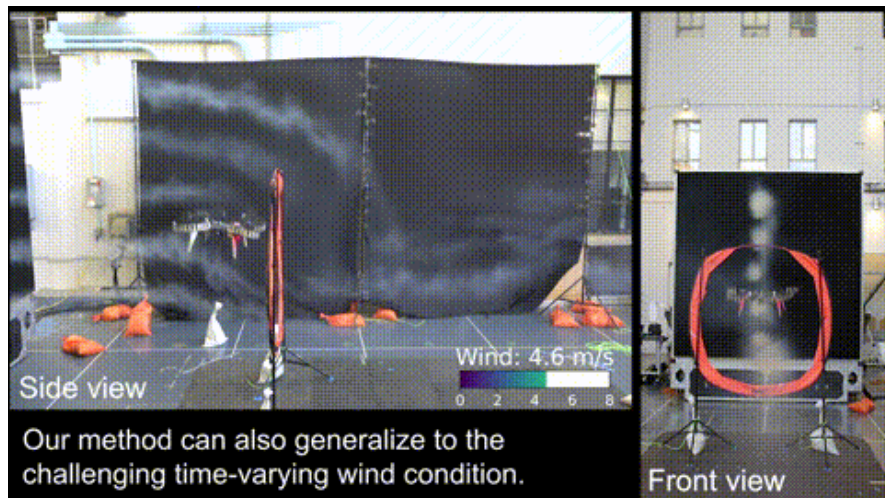
x : state
 u : action
 c : cost



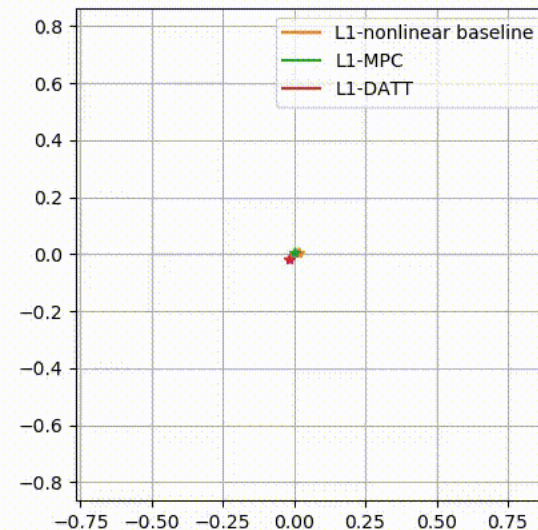
Lev Pontryagin

Model-based Control

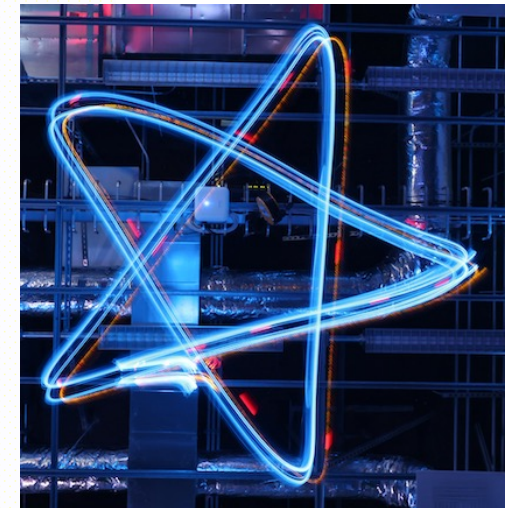
- ❑ Given the dynamics model $\dot{x} = f(x, u)$ or $x_{t+1} = f(x_t, u_t)$
- ❑ Design a *feedback* policy $u = \pi(x)$ achieving some goals:
 - Stabilization/regularization to some reference point x_d
 - Track a desired trajectory $x_d(t)$
 - **Minimize some cost functions – it is where RL started!**
 - Adaptive to uncertainty
 - Robust to disturbance



Neural-Fly, *Science Robotics*, 2022



DATT: Deep Adaptive Trajectory Tracking, CoRL (oral), 2023



Model-based RL and Optimal Control

- MBRL and optimal control are essentially both solving:
- c_t is cost function
 - X_t is the state constraint (e.g., collision avoidance)
 - U_t is the control constraint (i.e., actuation limit)
 - T is the horizon
 - Can be fixed and finite, infinite, or a decision variable
 - Become a time-optimal control problem when T is a decision variable

$$\begin{array}{ll} \min_{x,u} & \sum_{t=1}^T c_t(x_t, u_t) \\ \text{s. t.} & \left\{ \begin{array}{l} x_{t+1} = f_t(x_t, u_t) \\ x_t \in X_t \\ u_t \in U_t \end{array} \right. \end{array}$$

Model-based RL and Optimal Control

- ❑ Open-loop trajectory optimization (planning):
 - Solve a trajectory $x_{1:T}, u_{1:T}$ for low-level feedback control
 - Can we directly execute $u_{1:T}$?
- ❑ Closed-loop policy optimization:
 - Find a policy $u = \pi(x)$ to minimize the cost
- ❑ MBRL is basically optimal control with an unknown model
- ❑ Dive deeper: 16-745 by Prof. Zac Manchester

$$\begin{aligned} & \min_{x,u} \sum_{t=1}^T c_t(x_t, u_t) \\ \text{s. t. } & \begin{cases} x_{t+1} = f_t(x_t, u_t) \\ x_t \in X_t \\ u_t \in U_t \end{cases} \end{aligned}$$

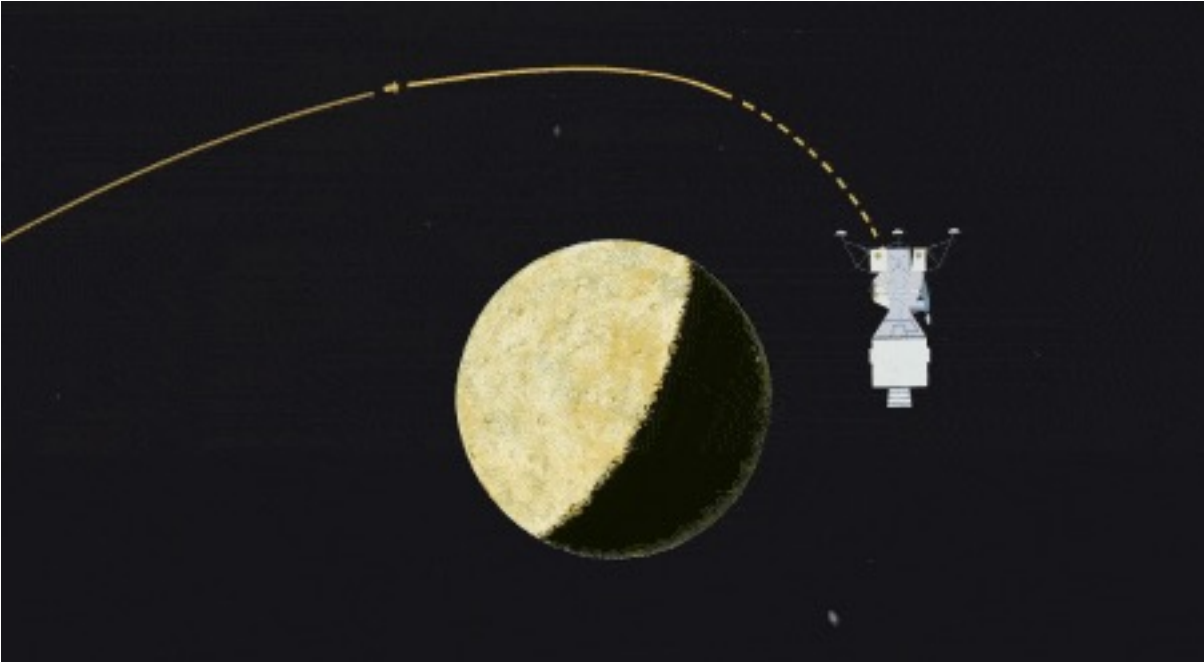
Optimal Control and Reinforcement Learning

16-745



Real-world Applications of Model-based RL and Control

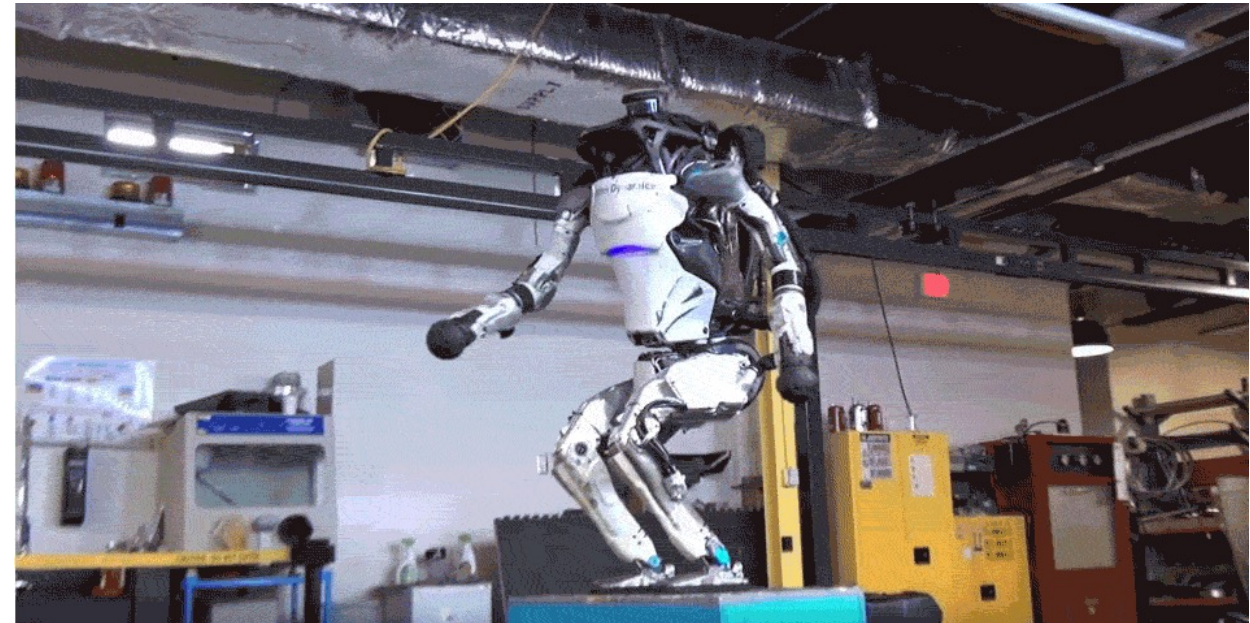
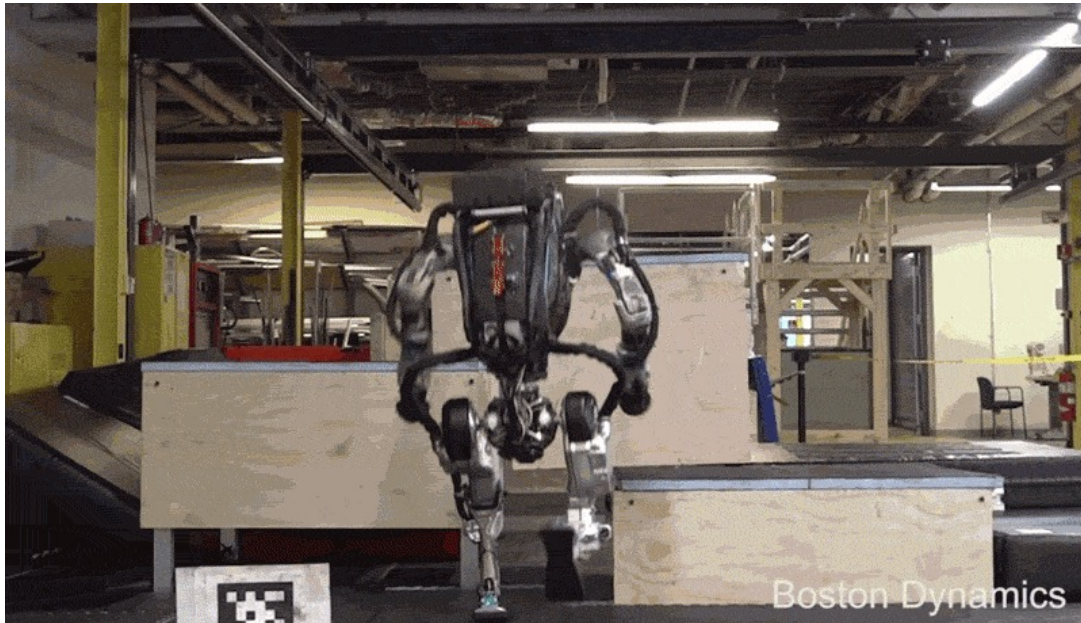
❑ W/o learning:



Apollo 11 trajectory optimization and control: optimal control + robust control

Real-world Applications of Model-based RL and Control

❑ W/o learning:



Boston Dynamics Atlas humanoid planning and control: trajectory optimization + MPC

Real-world Applications of Model-based RL and Control

❑ W/o learning:



Offroad autonomy: sampling-based MPC

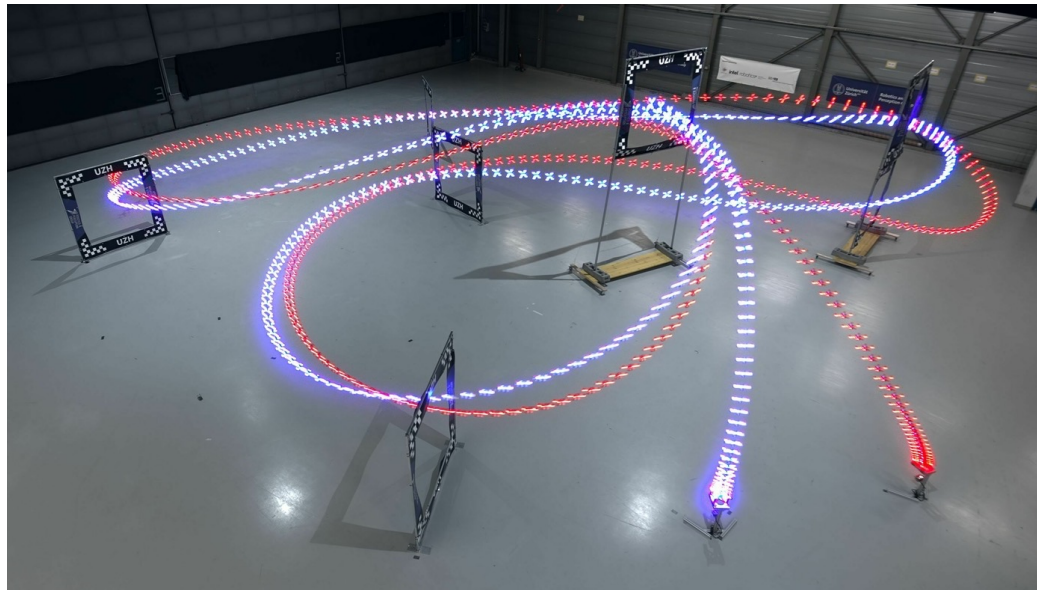
Model Predictive Control for Aggressive Driving Over Uneven Terrain

Tyler Han*, Alex Liu, Anqi Li, Alexander Spitzer, Guanya Shi, and Byron Boots

Real-world Applications of Model-based RL and Control

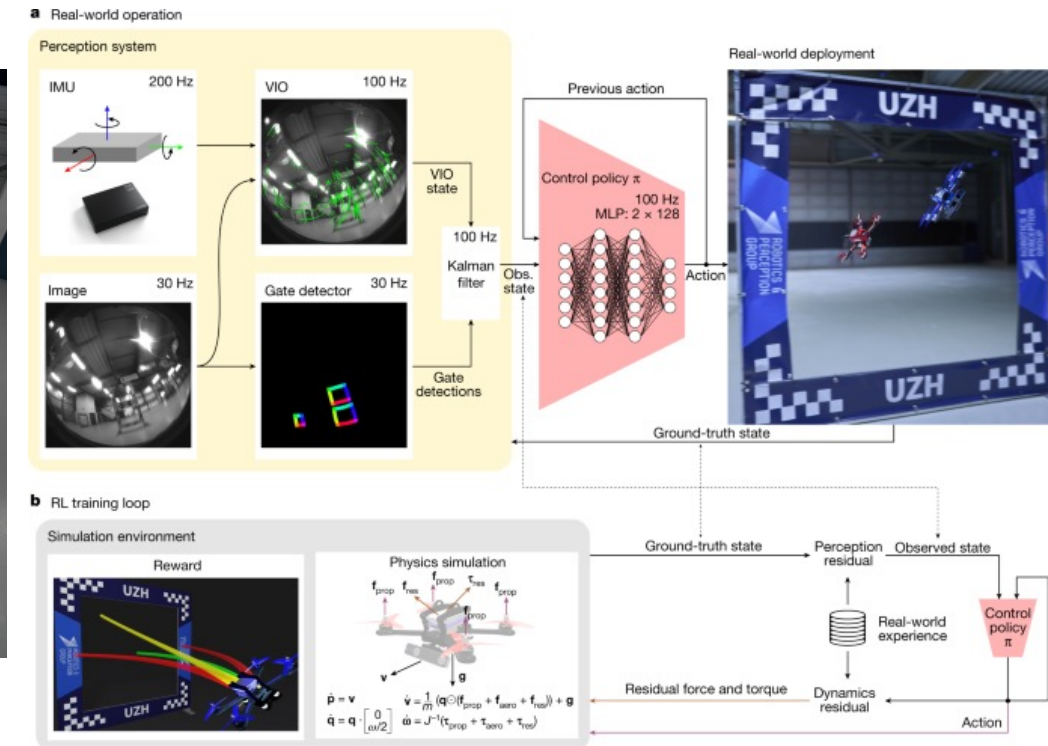
❑ W/ learning:

- Drone racing: learning both perception residual and dynamics residual



Champion-level drone racing using deep reinforcement learning

[Elia Kaufmann](#) , [Leonard Bauersfeld](#), [Antonio Loquercio](#), [Matthias Müller](#), [Vladlen Koltun](#) & [Davide Scaramuzza](#)

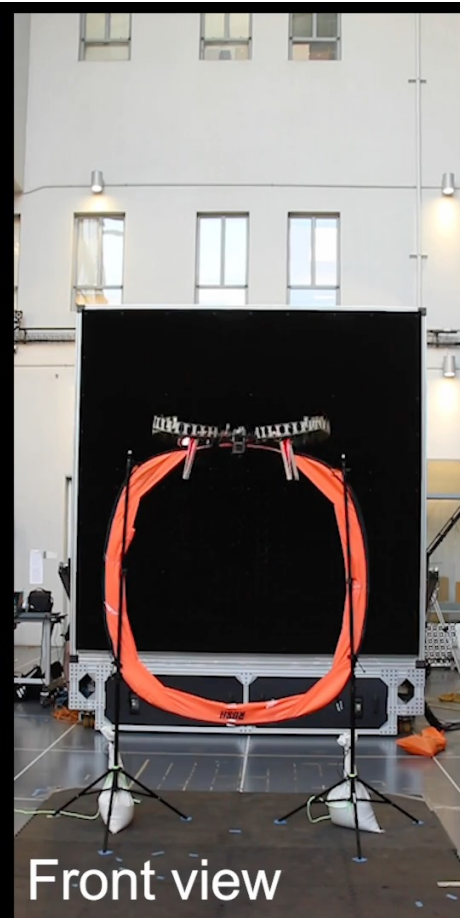


Real-world Applications of Model-based RL and Control

- ❑ W/ learning:
 - Fast adaptation in time-variant wind condition
 - Meta-learned dynamics model + adaptive control



Our method can also generalize to the challenging time-varying wind condition.



Neural-Fly Enables Rapid Learning for Agile Flight in Strong Winds

Michael O'Connell[†], Guanya Shi[†], Xichen Shi,
Kamyar Azizzadenesheli, Anima Anandkumar, Yisong Yue, Soon-Jo Chung*

Division of Engineering and Applied Science, California Institute of Technology

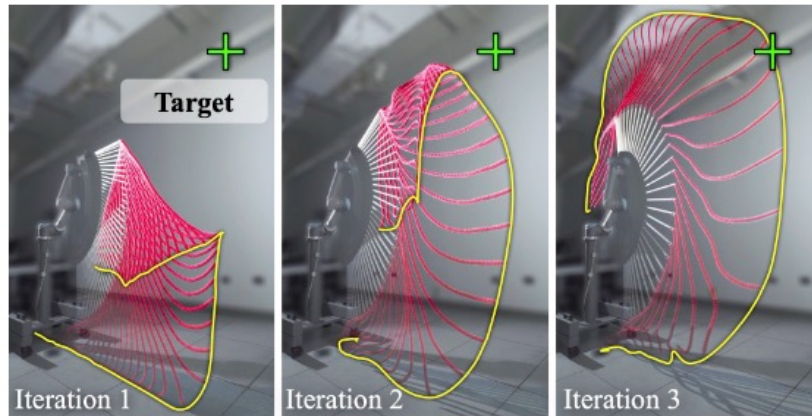
[†] The first two authors contributed equally to this article. Alphabetical order.

* Corresponding authors. Email: sjchung@caltech.edu

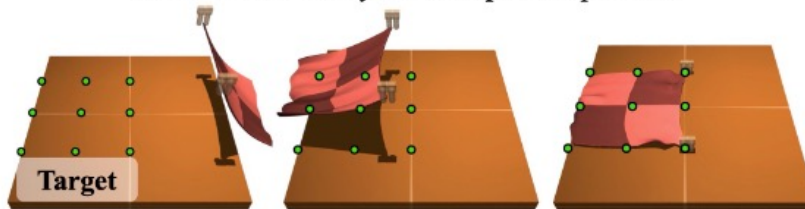
This is the accepted version of *Science Robotics* Vol. 7, Issue 66, eabm6597 (2022)
DOI: 10.1126/scirobotics.abm6597 Video: <https://youtu.be/TuF9teCZX0U>
Data and training code: <https://github.com/aerorobotics/neural-fly>

Real-world Applications of Model-based RL and Control

- ❑ W/ learning:
 - Whipping the rope to a target point
 - Learned a “delta” dynamics model



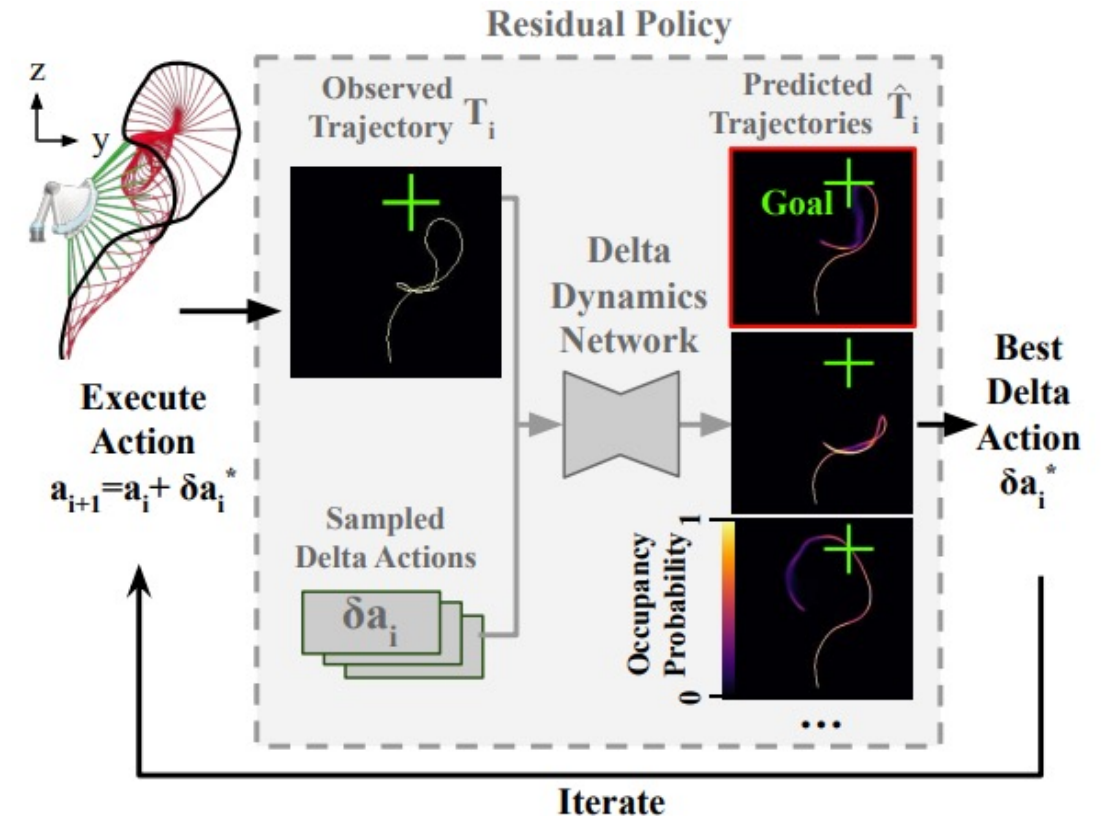
Goal-conditioned Dynamic **Rope** Manipulation



Goal-conditioned Dynamic **Cloth** Manipulation

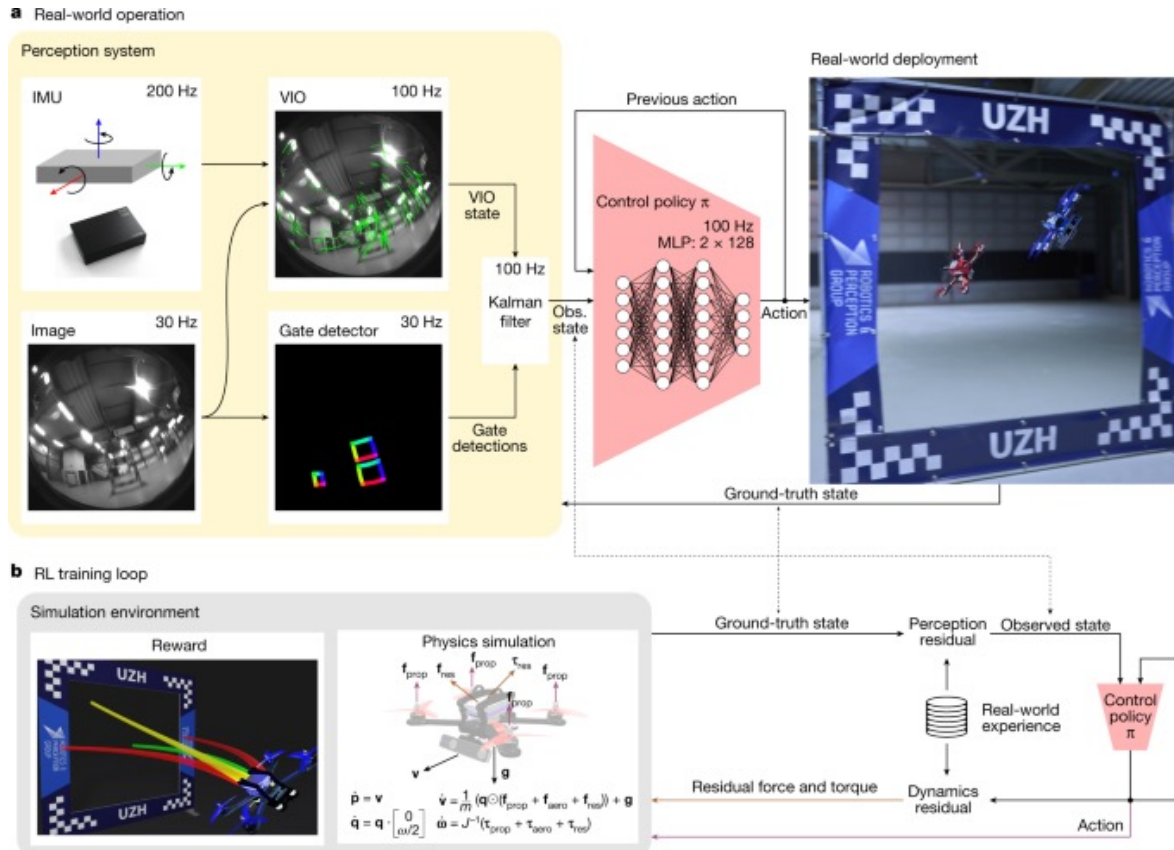
Iterative Residual Policy

for Goal-Conditioned Dynamic Manipulation of Deformable Objects



MFRL + MBRL/Control

- ❑ Why do we have to choose one from MFRL and MBRL/control?
 - We can do both!
- ❑ Example 1: learn a simulator model and do MFRL
 - Real (collect data and learn the model) -> Sim (MFRL) -> Real (deployment)



Philosophical question: is MFRL + sim2real MBRL or MFRL?

MFRL + MBRL/Control

- ❑ Why do we have to choose one from MFRL and MBRL/control?
 - We can do both!
- ❑ Example 2: hybrid MFRL and MBRL/control

Algorithm 1 Residual reinforcement learning

Require: policy π_θ , hand-engineered controller π_H .

```
1: for  $n = 0, \dots, N - 1$  episodes do
2:   Initialize random process  $\mathcal{N}$  for exploration
3:   Sample initial state  $s_0 \sim E$ .
4:   for  $t = 0, \dots, H - 1$  steps do
5:     Get policy action  $u_t = \pi_\theta(s_t) + \mathcal{N}_t$ .
6:     Get action to execute  $u'_t = u_t + \pi_H(s_t)$ .
7:     Get next state  $s_{t+1} \sim p(\cdot | s_t, u'_t)$ .
8:     Store  $(s_t, u_t, s_{t+1})$  into replay buffer  $\mathcal{R}$ .
9:     Sample set of transitions  $(s, u, s') \sim \mathcal{R}$ .
10:    Optimize  $\theta$  using RL with sampled transitions.
11:   end for
12: end for
```

Residual Reinforcement Learning for Robot Control

Tobias Johannink^{*1,3}, Shikhar Bahl^{*2}, Ashvin Nair^{*2}, Jianlan Luo^{1,2}, Avinash Kumar¹,
Matthias Loskyll¹, Juan Aparicio Ojea¹, Eugen Solowjow¹, Sergey Levine²

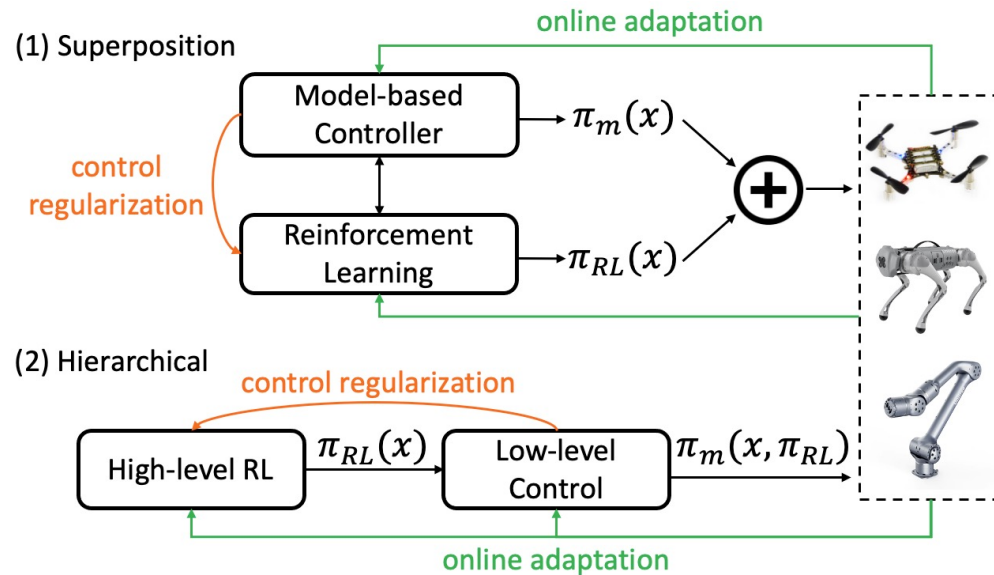


MFRL + MBRL/Control

❑ Why do we have to choose one from MFRL and MBRL/control?

- We can do both!

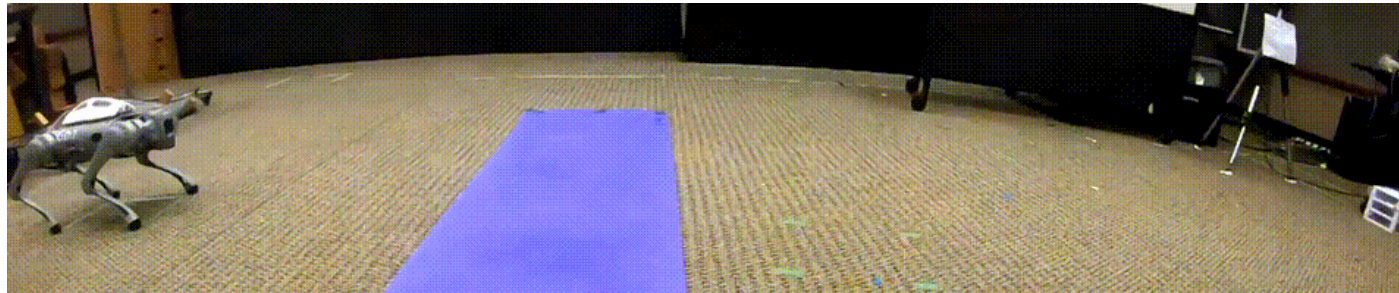
❑ Example 2: hybrid MFRL and MBRL/control



CAJun: Continuous Adaptive Jumping using a Learned Centroidal Controller

Yuxiang Yang*, Guanya Shi[†], Xiangyun Meng*, Wenhao Yu[‡], Tingnan Zhang[‡]
Jie Tan[‡], Byron Boots*

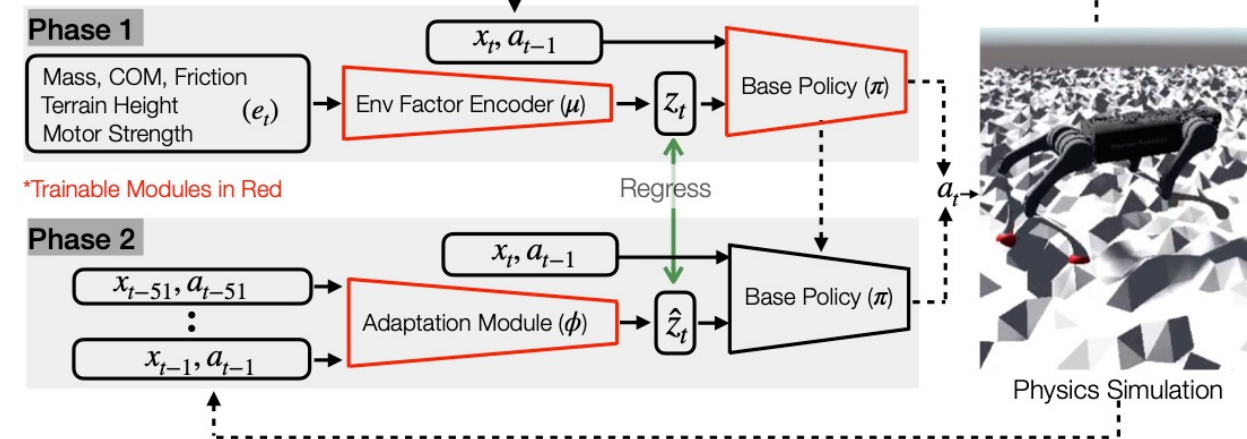
*University of Washington [†]Carnegie Mellon University [‡]Google Deepmind



MFRL + MBRL/Control

- ❑ Why do we have to choose one from MFRL and MBRL/control?
 - We can do both!
- ❑ Example 3: MFRL is conditioned on or augmented by model or MBRL/control

A) Training in Simulation



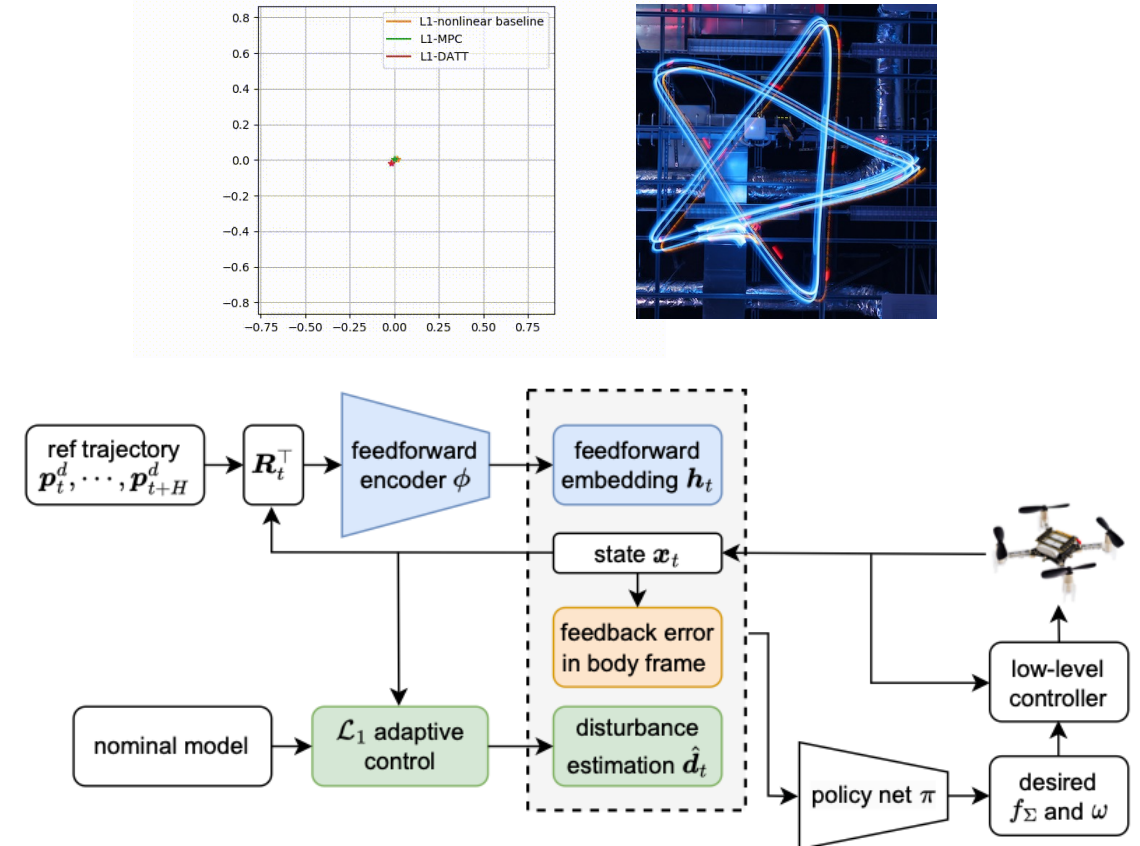
RMA: Rapid Motor Adaptation for Legged Robots

Ashish Kumar
UC Berkeley

Zipeng Fu
Carnegie Mellon University

Deepak Pathak
Carnegie Mellon University

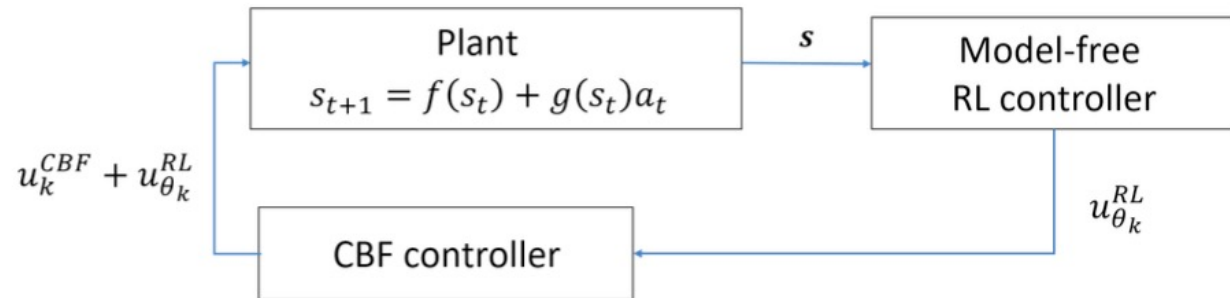
Jitendra Malik
UC Berkeley, Facebook



DATT: Deep Adaptive Trajectory Tracking, CoRL (oral), 2023

MFRL + MBRL/Control

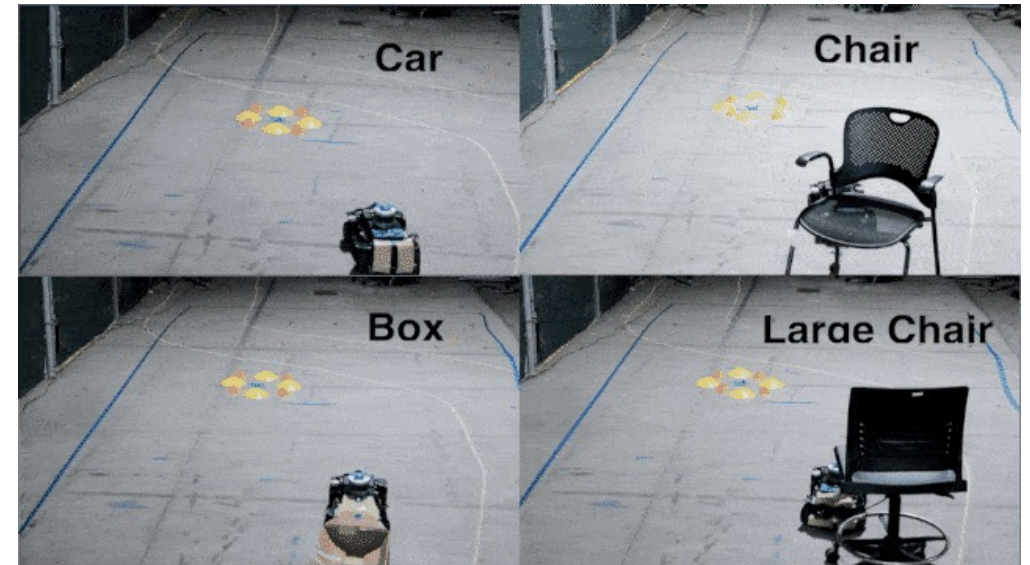
- ❑ Why do we have to choose one from MFRL and MBRL/control?
 - We can do both!
- ❑ Example 4: MFRL is filtered/guided/guarded by MBRL/control
 - For instance, MBRL/control for safety guarantees



End-to-End Safe Reinforcement Learning through Barrier Functions for Safety-Critical Continuous Control Tasks

Richard Cheng,¹ Gábor Orosz,² Richard M. Murray,¹ Joel W. Burdick¹

¹California Institute of Technology, ²University of Michigan, Ann Arbor



Safe Deep Policy Adaptation

Wenli Xiao*, Tairan He*, John Dolan, and Guanya Shi

Outline of the MBRL Topic Group

- ❑ Part 1: control theory basics
 - Stability and Lyapunov methods for stability
 - Uncertainty and robust control
 - Adaptive control
 - Linear systems and continuous-time LQR
 - Linearization of nonlinear systems
- ❑ Part 2: how can we make decisions if we *know* the dynamics?
 - Model predictive control (MPC)
 - Discrete time LQR
 - Sequential quadratic/convex programming (SQP/SCP)
 - iLQR/DDP
 - Sampling-based MPC
- ❑ Part 3: learn *unknown* dynamics and use it for control
 - Model learning
 - Policy optimization with the learned model
 - Deep MBRL: Dreamer, MBRL+MFRL, etc

Outline of the MBRL Topic Group

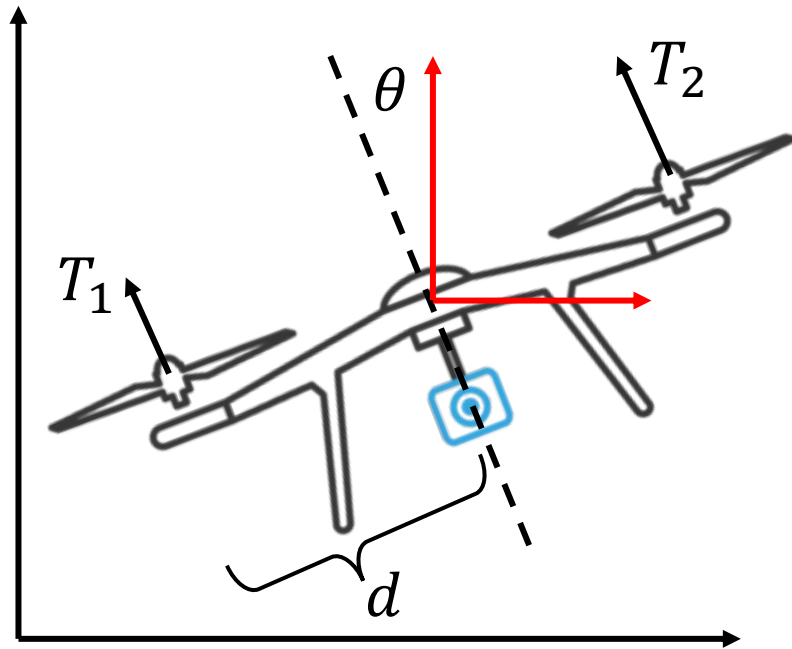
- ❑ Part 1: control theory basics
 - Stability and Lyapunov methods for stability
 - Uncertainty and robust control
 - Adaptive control
 - Linear systems and continuous-time LQR
 - Linearization of nonlinear systems
- ❑ Part 2: how can we make decisions if we *know* the dynamics?
 - Model predictive control (MPC)
 - Discrete time LQR
 - Sequential quadratic/convex programming (SQP/SCP)
 - iLQR/DDP
 - Sampling-based MPC
- ❑ Part 3: learn *unknown* dynamics and use it for control
 - Model learning
 - Policy optimization with the learned model
 - Deep MBRL: Dreamer, MBRL+MFRL, etc

Control Theory Basics

- ❑ Given the dynamics model $\dot{x} = f(x, u)$ or $x_{t+1} = f(x_t, u_t)$
- ❑ Design a *feedback* policy $u = \pi(x)$ achieving some goals:
 - Stabilization/regularization to some reference point x_d
 - Track a desired trajectory $x_d(t)$
 - Minimize some cost functions
 - Adaptive to uncertainty
 - Robust to disturbance
- ❑ Control theory: rigorously define and analyze these “goals”
- ❑ Today we will focus on the continuous-time model $\dot{x} = f(x, u)$
 - Why? (1) Easier to define concepts such as stability. (2) The physical world is continuous!

$\dot{x} = f(x, u)$: 2-d Drone Example

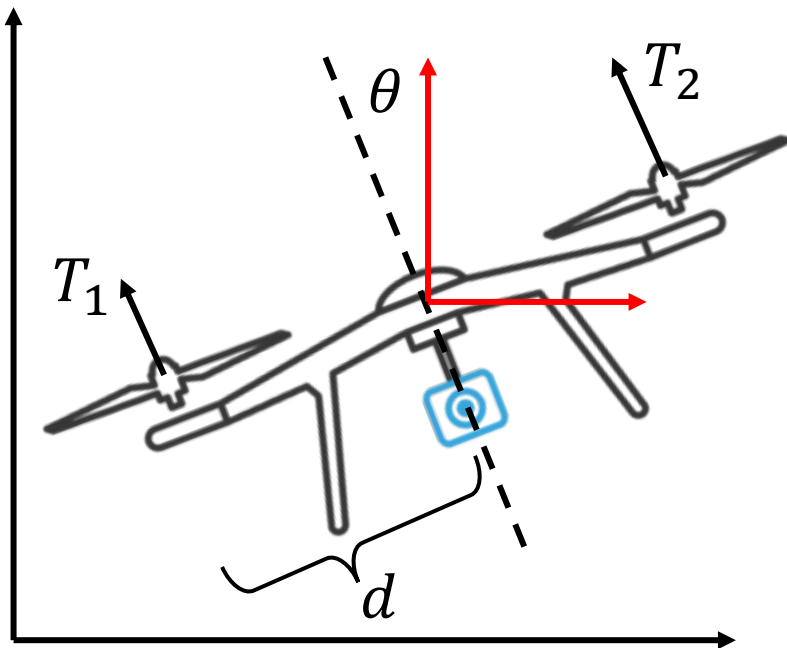
- ❑ Control and state are $x = [p; v; \theta; \omega] \in \mathbb{R}^6, u = [T; \tau] \in \mathbb{R}^2$
- ❑ $\vec{g} = [0; -g]$ is the gravity vector
- ❑ Control: total thrust $T = T_1 + T_2$, torque $\tau = (T_2 - T_1) \cdot d$
- ❑ Hovering equilibrium point: $v = 0, \theta = 0, \omega = 0, T = g \approx 9.81, \tau = 0$



$$\left\{ \begin{array}{l} \text{Translational kinematics:} \\ \dot{p} = v \\ \text{Translational dynamics:} \\ m\dot{v} = m\vec{g} + T \begin{bmatrix} -\sin\theta \\ \cos\theta \end{bmatrix} \\ \text{Rotational kinematics:} \\ \dot{\theta} = \omega \\ \text{Rotational dynamics:} \\ J\dot{\omega} = \tau \end{array} \right.$$

$\dot{x} = f(x, u)$: 2-d Drone Example

- Intuitively understand these concepts before diving into math:
- Stability: **If the drone is perturbed from equilibriums, can it come back?**
 - Uncertainty and robust control: **What if the model is wrong and “not learnable”?**
 - Adaptive control: **What if the model is wrong but “learnable”?**
 - Linearization of nonlinear systems: **Linearize the nonlinear term $T \begin{bmatrix} -\sin\theta \\ \cos\theta \end{bmatrix}$**
 - Linear systems and LQR: **Design an optimal policy for the linearized model**



Translational kinematics:

$$\dot{p} = v$$

Translational dynamics:

$$m\dot{v} = m\vec{g} + T \begin{bmatrix} -\sin\theta \\ \cos\theta \end{bmatrix}$$

Rotational kinematics:

$$\dot{\theta} = \omega$$

Rotational dynamics:

$$J\dot{\omega} = \tau$$

Control Basics: Stability

- ❑ Closed-loop system: $\dot{x} = f(x, \pi(x)) = f_{CL}(x)$
- ❑ Stability:

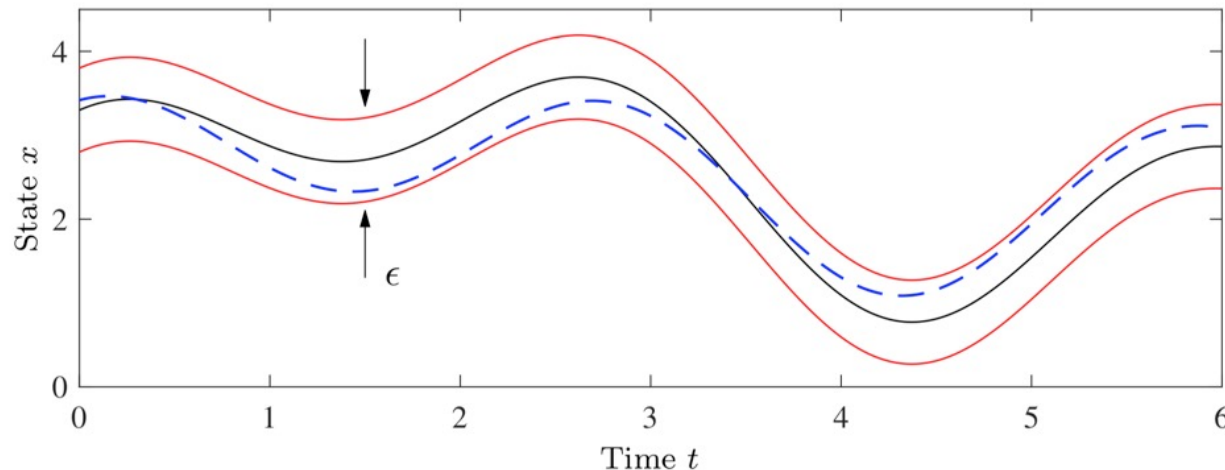
Definition (stable)

Let $x(t, a)$ be a solution to $f_{CL}(x)$ with $x(0) = a$. A solution is *stable* if other solutions that start near a stay close to $x(t, a)$. For all $\epsilon > 0$, there exists a $\delta > 0$ such that

$$\forall b, \|b - a\| < \delta \Rightarrow \|x(t, b) - x(t, a)\| < \epsilon, \forall t$$

When $x = a$ is an equilibrium point ($x(t, a) \equiv a$):

$$\forall b, \|b - a\| < \delta \Rightarrow \|x(t, b) - a\| < \epsilon, \forall t$$



Control Basics: Stability

Definition (asymptotically stable)

For all $\epsilon > 0$, there exists a $\delta > 0$ such that

$$\forall b, \|b - a\| < \delta \implies \|x(t, b) - x(t, a)\| < \epsilon, \forall t \text{ and} \\ \lim_{t \rightarrow \infty} \|x(t, b) - x(t, a)\| = 0$$

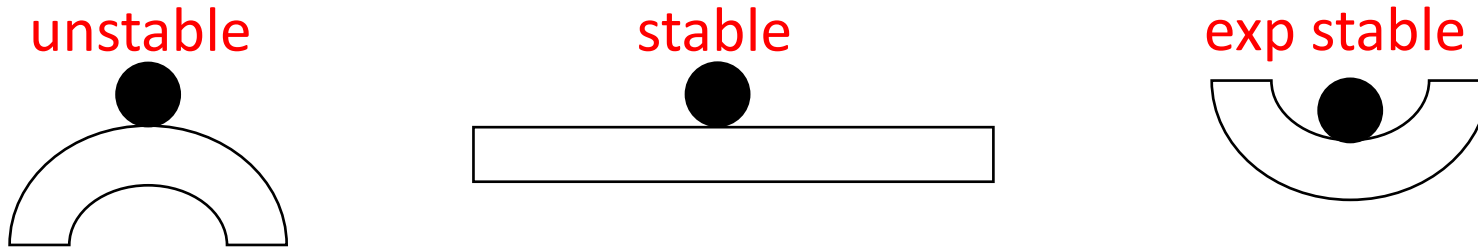
Definition (exponentially stable)

Asymptotically stable and

$$\|x(t, b) - x(t, a)\| < \alpha \|b - a\| e^{-\beta t}, \forall t$$

- ❑ Asymptotically stable: convergence
- ❑ Exponentially stable: fast (exponential) convergence
- ❑ For linear systems, these two types are equivalent.

Stability: Examples



Single integrator dynamics $\dot{x} = u$ and equilibrium point $x = 0$

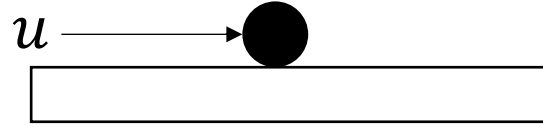
❑ Policy (a): $u = 0$

- Closed-loop dynamics $\dot{x} = 0$
- It is stable: the ODE solution is $x(t) = x(0)$

❑ Policy (b): $u = -kx$

- Closed-loop dynamics $\dot{x} = -kx$
- It is exponentially stable: the ODE solution is $x(t) = e^{-kt}x(0)$
- Role of feedback: this controller is called **proportional controller** (P controller)

Stability: Examples (Double Integrator)



Double integrator dynamics $\ddot{x} = u$ and equilibrium point $x = 0, \dot{x} = 0$

❑ Policy (a): $u = 0$

- Closed-loop dynamics $\ddot{x} = 0$
- It is NOT stable: the ODE solution is $x(0) = 0, \dot{x}(0) = \epsilon, x(t) = \epsilon t$

❑ Policy (b): $u = -x$

- Closed-loop dynamics $\ddot{x} = -x$
- It is (only) stable: the ODE solution is $x(t) = x(0)\cos(t) + \dot{x}(0)\sin(t)$

❑ Policy (c): $u = -x - 2\dot{x}$

- Closed-loop dynamics $\ddot{x} = -x - 2\dot{x}$
- Exponentially stable
- The ODE solution is $x(t) = x(0)e^{-t} + (x(0) + \dot{x}(0))te^{-t}$
- This controller is called **proportional-derivative controller** (PD controller)
- Damping terms for energy dissipation.

Lyapunov Methods for Stability

- ❑ Stability analysis in high-dimensional space is tricky (you don't want to solve ODEs in high-dimensional space and check its stability)
- ❑ Lyapunov function $V(x): \mathbb{R}^n \rightarrow \mathbb{R}_{\geq 0}$ is an energy-like positive definite function that can provide *sufficient* condition for stability

Stability via Lyapunov function (informal)

$$\dot{V} \leq 0$$

stable

$$\dot{V} < 0$$

asymptotically stable

$$\dot{V} \leq -\alpha V$$

exponentially stable

- ❑ Example: single integrator system $\dot{x} = u$ ($x, u \in \mathbb{R}^n$) with a P controller $u = -Kx$
 - Consider the Lyapunov function $V(x) = \frac{1}{2}x^\top x$
 - $\dot{V} = x^\top \dot{x} = -x^\top Kx$
 - $\dot{V} \leq -2\lambda_{\min}(K)V$
 - Therefore, exponentially stable if K is positive definite!

Uncertainty, ISS, and Integral Control

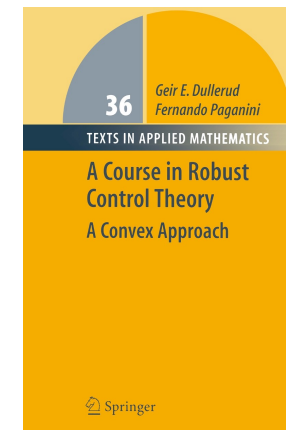
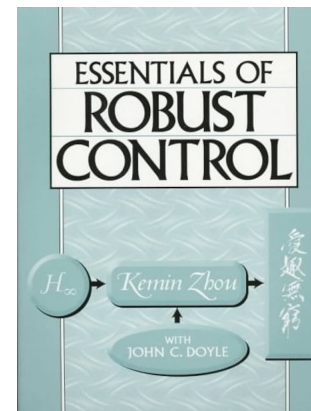
$$\dot{x} = f(x, u) + d$$

- ❑ d is disturbance on top of the “nominal” model $f(x, u)$
 - d could be time-variant or state/action-dependent
- ❑ Goal of robust control: robustness to *bounded* disturbance
- ❑ Input-to-State Stability (ISS)
 - Not converge to exactly 0, but some error ball proportional to error

Definition (input-to-state stability, informal)

$$\|x(t, b) - x(t, a)\| < \alpha \|b - a\| e^{-\beta t} + \gamma \|b - a\|, \forall t$$

- ❑ More to read: robust control textbook



Uncertainty, ISS, and Integral Control

Example: disturbed single integrator dynamics $\dot{x} = u + d$

□ P controller: $u = -kx$

- Closed-loop dynamics $\dot{x} = -kx + d$
- The ODE solution is $x(t) = e^{-kt}x(0) + \frac{d}{k}(1 - e^{-kt})$
- It is ISS!
- It why *exponential stability* is important: it implies robustness.

□ How to get rid of d : $u = -2x - \int_t x$

- Closed-loop dynamics $\dot{x} = -2x - \int_t x + d$
- Taking derivatives of both sides: $\ddot{x} = -2\dot{x} - x,$
- It is exponentially stable!
- This controller is called **proportional-integration controller** (PI controller)
- Intuition: integral control deals with “constant” or “slow-varying” disturbance
- Most simple adaptive control

Adaptive Control

□ General recipe:

- a : unknown system parameters; $\hat{a}$: estimation

	Continuous	Discrete
System	$\dot{x} = f(x, u, a(t))$	$x_{t+1} = f(x_t, u_t, a_t)$
Policy	$u = \pi(x, \hat{a}; \theta_\pi)$	$u_t = \pi(x_t, \hat{a}_t; \theta_\pi)$
Adaptation	$\dot{\hat{a}} = \mathcal{A}(\hat{a}(0:t), x(0:t), u(0:t); \theta_{\mathcal{A}})$	$\hat{a}_{t+1} = \mathcal{A}(\hat{a}_{1:t}, x_{1:t}, u_{1:t}; \theta_{\mathcal{A}})$

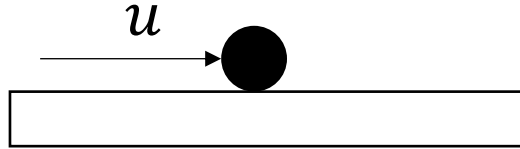
□ MIAC: model identification adaptive control (e.g., $\mathcal{L}_1$ adaptive control)

- Perform system ID while the system is running
- Goal: $\hat{a} \rightarrow a$ in a “good” way
- More intuitive, but stability and robustness are tricky

□ MRAC: model reference adaptive control (e.g., integral control)

- Incorporate an “ideal” model defining desired closed-loop performance
- MRAC does NOT directly do system ID
- $\hat{a} \rightarrow a$ is neither necessary nor important

Trajectory Tracking Controller



Double integrator dynamics $\ddot{x} = u$. The goal is to track a trajectory $x_d(t)$

□ Tracking controller:

$$u = -K_P(x - x_d) - K_D(\dot{x} - \dot{x}_d) + \ddot{x}_d \rightarrow \text{feedforward term}$$

□ PD feedback + feedforward (desired acceleration)

□ $\dot{x}_d, \ddot{x}_d = 0$ for setpoint tracking!

□ Why we need the feedforward term?

- Define tracking error as $e = x - x_d$
- Then we have the error dynamics:

$$\begin{aligned}\ddot{x} &= -K_P e - K_D \dot{e} + \ddot{x}_d \\ \ddot{e} + K_P e + K_D \dot{e} &= 0\end{aligned}$$

- Intuition: predicting future

Linear Systems and LQR

- ❑ $x \in \mathbb{R}^n$: state of the robot; $u \in \mathbb{R}^m$: control input
- ❑ $\dot{x} = Ax + Bu$
- ❑ Linear policy $u = -Kx$ where $K \in \mathbb{R}^{m \times n}$ is called gain matrix
- ❑ Closed-loop system:

$$\dot{x} = (A - BK)x = A_{CL}x$$

- ❑ Exponentially stable if A_{CL} is a *Hurwitz matrix*
 - I.e., all eigenvalues have strictly negative real parts
 - Stable if only non-positive
- ❑ How to design K ? Gain tuning or LQR!

Linear Systems and LQR

- $x \in \mathbb{R}^n$: state of the robot; $u \in \mathbb{R}^m$: control input
- Linear quadratic regulator (LQR):

$$\min_u J = \int_0^\infty x^\top Q x + u^\top R u$$

subjective to $\dot{x} = Ax + Bu$

- Closed-form solution:

$$u = -K^* x$$
$$K^* = R^{-1} B^\top P$$
$$A^\top P + PA + Q = PBR^{-1}B^\top P \xrightarrow{\text{ARE}}$$

- Derived using dynamic programming (HJB) or calculus of variations (PMP)
- You don't have to solve it – just call LQR functions in Python or Matlab
- P is a positive definite matrix solved from the algebraic Riccati equation (ARE)
- One can prove that $(A - BK^*)$ is Hurwitz (under mild assumptions on A, B, Q, R)
– LQR controller is not only optimal, but also stable!
- Next lecture: detailed derivation in the discrete case using dynamic programming

Linearization of Nonlinear Models

- $x \in \mathbb{R}^n$: state of the robot; $u \in \mathbb{R}^m$: control input
- For a nonlinear system $\dot{x} = f(x, u)$ with an equilibrium point x_e, u_e ($f(x_e, u_e) = 0$)
- Linearization around x_e, u_e

$$A = \left. \frac{\partial f}{\partial x} \right|_{x=x_e, u=u_e}, B = \left. \frac{\partial f}{\partial u} \right|_{x=x_e, u=u_e}$$

- Then $\dot{x} \approx A(x - x_e) + B(u - u_e)$ near the equilibrium point
- If x_e, u_e is not an equilibrium point: $\dot{x} \approx A(x - x_e) + B(u - u_e) + f(x_e, u_e)$
 - Will use it in the next lecture for SQP/SCP/iLQR/DDP

Example: 2d Drone Linearization

- Hovering equilibrium point: $v = 0, \theta = 0, \omega = 0, T = g, \tau = 0$
- $x = [p; v; \theta; \omega] \in \mathbb{R}^6, u = [T; \tau] \in \mathbb{R}^2$
- Linearization:

$$\frac{\partial f}{\partial x} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & -T \cos \theta & 0 \\ 0 & 0 & 0 & 0 & -T \sin \theta & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \frac{\partial f}{\partial u} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ -\sin \theta & 0 \\ \cos \theta & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix}$$

- Plug in the equilibrium point:

$$\begin{cases} \dot{p} = v \\ \dot{v} = \begin{bmatrix} -g\theta \\ T - g \end{bmatrix} \\ \dot{\theta} = \omega \\ \dot{\omega} = \tau \end{cases}$$

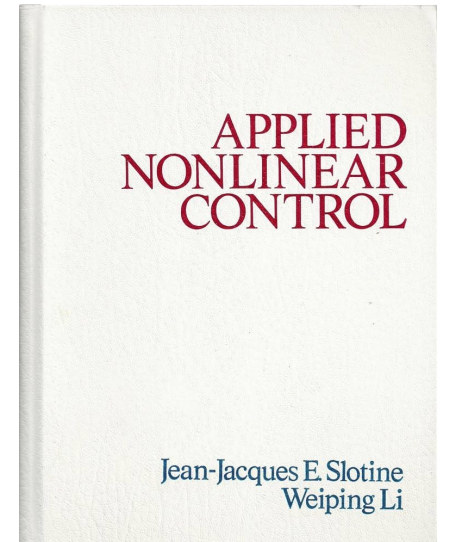
$$\begin{cases} \text{Translational kinematics:} \\ \dot{p} = v \\ \text{Translational dynamics:} \\ \dot{v} = \vec{g} + T \begin{bmatrix} -\sin \theta \\ \cos \theta \end{bmatrix} \\ \text{Rotational kinematics:} \\ \dot{\theta} = \omega \\ \text{Rotational dynamics:} \\ \dot{\omega} = \tau \end{cases}$$

Feedback Linearization

- ❑ Another method is to “cancel out” the nonlinearity
- ❑ A lot of theory, will not cover in this course
- ❑ Basic idea (using a linear system with matched uncertainty as an example):

$$\dot{x} = Ax + B(u + g(x))$$

- ❑ $g(x)$ is the nonlinear term. It could be anything (neural network, LLM) but *known*.
- ❑ Consider the auxiliary control $u' = u + g(x)$
- ❑ Then the system is *linear* w.r.t. u' : $\dot{x} = Ax + Bu'$
- ❑ Idea: we design a controller for u' first, and then “map” it back to u
 - For example, LQR for u'
- ❑ More to read: any nonlinear control textbook



Thank You!